

SAGA Authorization Study: Paper Outline

Abstract

1 Introduction

1.1 Motivation

1.2 Problem

1.3 Key Observation

1.4 Contributions

2 Background and SAGA Overview

2.1 System Model

2.2 Agent Registration

2.3 Contact Policy

2.4 Agent Access Protocol

2.5 Access-Control Token

2.6 Existing Formal Security Claims

3 Security Gap and Problem Formulation

3.1 Authentication Is Not Authorization

SAGA uses identity checks and access-control tokens to govern contact between agents. These mechanisms answer different questions. Authentication establishes which participant completed a protocol step; token validation establishes that a presented token satisfies the checks performed by the receiving agent. Authorization additionally asks whether the receiving agent’s user-defined contact policy permits the initiating agent to contact it. A credential can be authentic and valid under the token checks while the decision to make that credential available disagrees with the policy’s intended meaning.

The reference communication model focuses on participant authentication and token secrecy. These guarantees concern the origin of protocol messages and the confidentiality of credentials; they do not, by themselves, establish that a contact complies with the receiving user’s policy. In particular, confidential delivery to an authenticated participant does not establish that the participant is entitled to receive a credential under that policy.

The missing requirement is an authorization basis for accepted contact, preserved through credential issuance and message acceptance. Chapter 4 examines implementation disagreements with policy meaning. This chapter defines the requirement without asserting an end-to-end access violation.

3.2 Policy–Authorization Consistency

We call the end-to-end requirement *policy–authorization consistency*: every accepted access must be traceable to an applicable authorization decision. For every execution and every accepted-access event e , we require

$$\text{AccessAccepted}(e) \implies \exists d. \text{AllowDecision}(d) \wedge \text{Justifies}(d, e).$$

In addition to this end-to-end property, we consider authorization-soundness conditions at intermediate protocol boundaries, including Provider grants and credential issuance. These intermediate events are not themselves instances of `AccessAccepted`.

Here, d is a decision occurring in that execution, and `AllowDecision`(d) means that it grants permission under the intended policy semantics. An implementation’s report of “allow” alone does not establish this predicate. The relation `Justifies`(d, e) requires that the decision precede the accepted access, cover the same subject, target, and, where applicable, operation,

and remain applicable under the relevant policy state. Where a credential mediates access, the decision must also be linked to the grant and credential used for that access. An unrelated earlier allow decision is not a sufficient justification. This is a general requirement whose concrete events and policy-state conditions must be fixed for each formal model.

Let I be an initiating agent, R a receiving agent, and P_R the contact rulebook selected by R 's user. Each rule associates an agent-identity pattern with a budget. We write $\text{Allow}_{\text{spec}}(P_R, I)$ for the decision prescribed by the intended policy semantics, rather than the value returned by any particular implementation of the matcher. In the cases considered here, the unique highest-scoring matching rule determines the budget; a positive budget permits contact, whereas a nonpositive budget or no matching rule does not. Rule selection follows the matcher's documented most-specific-rule contract and its AID specificity score; the positive-budget convention follows the Provider's access check.¹ We initially fix the rulebook throughout an execution and consider either no matching rule or a unique highest-scoring match. Equal-score conflicts require an explicit convention and are outside this initial scope.

We distinguish three analysis-level events. $\text{ProviderGrant}(I, R)$ denotes the Provider's release, through its access interface, of the material needed for I to continue the contact protocol with R . The event $\text{TokenIssued}(R, I, T)$ denotes R 's creation of token T for I and its release for delivery through the protocol; it does not assert that I has received or used T . Finally, $\text{MessageAccepted}(R, I, T, m)$ denotes R 's acceptance of an application message m from I with token T , after the receiving agent's access checks, for application processing. It does not assert the completion of a downstream tool action.

Under this static-policy scope, the intermediate condition at the Provider boundary is

$$\text{ProviderGrant}(I, R) \implies \text{Allow}_{\text{spec}}(P_R, I),$$

whereas a necessary permission condition for end-to-end consistency at the final message-acceptance boundary is

$$\text{MessageAccepted}(R, I, T, m) \implies \text{Allow}_{\text{spec}}(P_R, I).$$

The intermediate condition at token issuance is

$$\text{TokenIssued}(R, I, T) \implies \text{Allow}_{\text{spec}}(P_R, I).$$

¹See `match()` and `aid_specificity()` in `saga/common/contact_policy.py`, and `access()` in `saga/provider/provider.py`.

All three implications range over every occurrence of the respective events and all their arguments. The intermediate conditions are separate obligations, not logical consequences of end-to-end consistency alone. These implications express permission at their respective boundaries; they do not replace the decision-to-access linkage required by **Justifies**. Nor does policy permission alone suffice for access: certificate checks, key availability, token validity, expiry, and quota may still prevent a permitted contact. These conditions address contact permission, not quantitative bounds on credential issuance or message use.

For a policy that can change over time, the applicable policy version and the effect of revocation on existing credentials must additionally be specified. Quota consumption and token expiry likewise require explicit state and validity conditions. The general requirement leaves room for these obligations, but the static permission conditions above do not establish them.

A message-level authorization claim therefore requires a model connecting policy evaluation, credential issuance, and the final message-acceptance event. Chapter 6 develops the corresponding formalization and states its assumptions.

3.3 Authorization Pipeline

The relevant decision is distributed across several stages. The user's rulebook determines an intended answer for I contacting R . The Provider evaluates its stored copy of that rulebook and, if the request passes its checks, returns one-time key material. The initiating and receiving agents then complete the credential exchange; the receiving agent generates a token and later validates a token presented with a message. An authorization argument must connect the same initiating identity and receiving identity across these stages, rather than infer permission solely from the presence of a valid token. The intended progression is

$$P_R \longrightarrow \text{ProviderGrant} \longrightarrow \text{TokenIssued} \longrightarrow \text{MessageAccepted}.$$

The arrows indicate protocol stages and authorization dependencies, not a guarantee that reaching one stage causes the next to succeed.

Each boundary requires separate evidence: a Provider response does not establish token issuance, and token issuance does not establish message acceptance. Later checks and protocol steps may fail.

When $\neg\text{Allow}_{\text{spec}}(P_R, I)$ holds, we distinguish three levels of violation:

1. **Provider decision violation:** $\text{ProviderGrant}(I, R)$ occurs despite the policy denial.

2. **Credential issuance violation:** $\text{TokenIssued}(R, I, T)$ occurs despite the policy denial. Issuance alone does not establish delivery or subsequent use.
3. **End-to-end authorization violation:** $\text{MessageAccepted}(R, I, T, m)$ occurs despite the policy denial. This is acceptance at the receiving agent’s application boundary.

These levels classify possible observations rather than assert results. A sound pipeline must preserve the link to the applicable decision, participants, and credential across stages. Chapter 6 specifies the corresponding events; Chapter 7 evaluates implementation observations and model results separately.

3.4 Threat Model

We consider a normally registered initiating agent controlled by the adversary. It can choose its contact requests and participate in the protocol under its genuine identity and certificate. The receiving user’s rulebook is an existing configuration: the adversary cannot change its contents or rule order, control the Provider or CA, or obtain another honest party’s private keys.

The Provider and receiving agent execute their implementations with the same fixed rulebook. Cryptographic checks are assumed effective. The failure under study is a disagreement between policy meaning and enforcement during ordinary protocol operation, without compromise of either enforcement point.

The adversary’s objective is an execution in which the intended policy denies contact by I , yet the system performs one of the authorization-relevant actions above. At the message level, the violation condition is

$$\text{MessageAccepted}(R, I, T, m) \wedge \neg \text{Allow}_{\text{spec}}(P_R, I).$$

This end-to-end violation is distinct from an erroneous Provider grant or credential issuance. Claiming it requires evidence that the intervening checks and protocol steps reach the receiving agent’s acceptance boundary.

We study this objective under a static policy. Policy updates and revocation, disagreement between stored policy versions, quantitative exhaustion of budgets or quotas, compromise of trusted parties, and arbitrary actions after contact require additional assumptions and are outside this initial question. The distinction between zero, negative, and positive configured budgets remains part of the contact-permission semantics.

4 Authorization Inconsistencies in SAGA

This chapter examines how the reference implementation interprets contact policy at its enforcement points. We distinguish the documented selection of a rule from the interpretation of the selected budget. The analysis concerns the fixed-policy setting of Chapter 3 and is based on source inspection; runtime propagation to the three security boundaries requires separate execution evidence.

4.1 Intended Contact-Policy Semantics

A contact rulebook is represented as an ordered list of rules, each carrying a `pattern` and an integer `budget`. A pattern matches an agent identifier, or AID. For contact from I to R , the relevant object is R 's rulebook evaluated against I 's AID: permission has a direction. Its evaluation does not distinguish individual application commands or tool actions. Throughout this chapter, *policy budget* denotes the `budget` field of a contact rule, not the communication quota associated with an access-control token.

In `contact_policy.py`, `check_rulebook()` checks rule fields and admits integer budgets from -1 upwards. The `match()` function first checks AID syntax and then tests patterns with `fnmatch.fnmatch()`. Its documentation states that the returned budget belongs to the most specific matching pattern. The function `aid_specificity()` supplies the comparison score, using weighted contributions from literals and wildcard constructs in the user and agent-name components. We use this numerical ordering as the documented selection criterion, without assuming that it is a general ordering by containment between pattern languages.

For a well-formed fixed rulebook with a unique highest-scoring match, let b^* denote the budget attached to that rule. For no match, take $b^* = 0$, consistent with the matcher's default result. The policy baseline is

$$\text{Allow}_{\text{spec}}(P_R, I) \iff b^* > 0.$$

The positivity convention comes from the Provider's `/access` handler, which rejects both negative and zero results. Thus the baseline combines a documented rule-selection contract with the Provider's explicit interpretation of the result. It does not assert a separate published policy language or resolve conflicts between equally specific rules. A malformed AID produces a separate error result and is outside the valid-identity cases considered here.

A positive policy decision is only permission to proceed through this check. The Provider additionally uses resource availability and remain-

ing contact allowance, and later protocol stages impose their own checks. Accordingly, this baseline specifies contact eligibility rather than sufficient conditions for token issuance or application-message acceptance.

4.2 Rule-Specificity Selection Flaw

The implementation of `match()` does not maintain the state needed by its documented selection rule. It initializes `best_pattern` to `None` and the return budget to zero. For each matching rule, it compares that rule’s specificity with `aid_specificity(best_pattern)` and updates the budget when the comparison succeeds. It never updates `best_pattern`. Consequently, later rules are compared with the initial sentinel, not with the most specific rule already encountered.

This is more precise than saying that specificity is ignored: the comparison is executed, but its reference value remains wrong. The score of `None` is -1 , whereas the scores computed for well-formed patterns are nonnegative, including zero for the universal wildcard. Therefore every matching rule handled normally by the scorer passes the comparison. For a valid AID and a well-formed rule list, the returned budget is the budget of the last matching rule encountered; with no match it remains zero.

The order involved is the rulebook’s list order, not the order of keys within each rule dictionary. The registration path carries the list in the `contact_rulebook` field, the Provider stores that field, and the Agent loads it into its local state. None of these paths sorts the rules by specificity. The matcher’s ordinary list traversal consequently preserves the distinction between earlier and later rules.

Consider two rules that both match the same valid identity I . Let r_s have a strictly higher specificity score and budget -1 , and let r_g have a lower score and budget 10 . This is an abstract policy example; it does not assume that I can edit R ’s configuration. The documented selection is r_s in either ordering. Inspection of the loop yields the following different implementation results:

Rule order	Intended budget	Returned budget
$[r_s, r_g]$	-1 (deny)	10
$[r_g, r_s]$	-1 (deny)	-1

To compare Boolean decisions, define

$$\text{Allow}_{\text{impl}}(P_R, I) \iff \text{match}(P_R, I) > 0.$$

This is the actual matcher result interpreted by the Provider’s policy check, not the success of the entire endpoint. For the first ordering above,

$$\text{Allow}_{\text{impl}}(P_R, I) \neq \text{Allow}_{\text{spec}}(P_R, I).$$

The same logical rule set thus has an order-dependent implementation result despite an unchanged unique maximum under the documented semantics. Equal scores are unnecessary for this disagreement. The observation establishes a policy-evaluation mismatch; it does not, by itself, establish any of the credential or message events defined in Chapter 3.

4.3 Budget-Semantics Inconsistency

The two enforcement points also differ in how they interpret an identical matcher result. In `provider.py`, `access()` returns a denial for `budget < 0` and separately for `budget == 0`. Only a positive value proceeds to the resource-allocation part of the handler. Writing `ProviderAllow` for this policy check alone gives

$$\text{ProviderAllow}(b) \iff b > 0.$$

This predicate excludes the endpoint’s identity, registration, and resource checks; a true value is not yet a Provider grant.

In `agent.py`, the incoming connection handler evaluates the local rule-book against the initiating AID using `match()`. It rejects only when the returned value is negative. The budget is not tested for equality to zero at this point. For this particular precheck,

$$\text{AgentPrecheckAllow}(b) \iff b \geq 0.$$

Here “allow” means only that this conditional does not reject the connection. It is neither a statement that a token is generated nor a statement that an application message is accepted.

Policy result	Provider policy check	Agent precheck
$b < 0$	Reject	Reject
$b = 0$	Reject	Continue
$b > 0$	Continue	Continue

The difference includes an explicit zero-budget match and the zero returned when no rule matches. At either source of zero, the Provider treats the result as insufficient permission, while the Agent’s precheck does not stop the connection. However, the ordinary fresh-contact path obtains material through the Provider first. Its zero-budget denial remains effective at

that boundary. The receiver-side difference therefore does not independently establish a complete unauthorized execution.

Several distinct quantities must be kept separate when assessing this behavior. The configured rule budget is an input to policy evaluation. The Provider’s **counter** stores a remaining allowance for each initiating AID within a target Agent’s record. A first allocation initializes the entry to the selected budget minus one; subsequent allocations decrement that entry, with a floor of zero. An exhausted entry blocks further allocation. Despite its name, this is remaining contact allowance, not an accumulated count of accepted application messages. Allocation consumes allowance before the later credential exchange has necessarily completed.

Token-use state is maintained separately. The **generate_token()** function sets the token’s **communication_quota** from the configured **Q_MAX** value, not from the matched contact-rule budget. Token validation checks the receiver’s active-token state, expiration, remaining token quota, and recipient access-control key. The conversation loop decrements token quota and also has a separate **MAX_QUERIES** guard. These checks concern token use and local conversation processing. Neither their presence nor their success establishes that the correct policy rule was selected, and the zero-budget comparison cannot be interpreted as a token-quota exhaustion test.

4.4 Cross-Layer Authorization Inconsistency

The findings concern two different relationships. The rule-selection flaw is a *specification-to-implementation inconsistency*: the matcher can return a budget that disagrees with the documented most-specific-rule contract. The zero-budget difference is an *enforcement-to-enforcement inconsistency*: two consumers of the same policy result apply different predicates. In the latter case,

$$\text{ProviderAllow}(0) \neq \text{AgentPrecheckAllow}(0).$$

The second disagreement does not depend on overlapping rules. Conversely, the first can change the sign of the returned budget without involving zero. They should not be assigned the same consequence merely because both concern authorization.

The Provider and receiving Agent both invoke the common matcher. The Agent’s incoming policy check therefore supplies an additional enforcement point, but it does not independently recover the documented rule selection. With the same ordered rulebook and identity, both invocations inherit the same selection error. Agreement between two implementations of a check

is insufficient when both rely on the same incorrect decision procedure. At zero, even their interpretation of that shared result differs.

These observations concern the preservation of policy meaning along the pipeline defined in Chapter 3. The Provider’s successful response supplies contact material, including one public one-time key and its signature, the target Agent’s certificate and public access-control material, and the target user’s certificate. It removes the rulebook and counter from that response. This response is not an access-control token; the receiving Agent generates the token at a later boundary.

The incoming handler performs its policy precheck before credential handling. Subsequent identity, Provider-stamp, signature, and one-time-key checks can still stop a new credential exchange. Their purpose differs from choosing the applicable contact rule. In particular, the Provider stamp authenticates an Agent card constructed at registration, rather than a decision selecting a rule for this contact. Later, `token_is_valid()` checks credential state without re-evaluating rule specificity. Thus successful cryptographic checks cannot be substituted for the missing semantic agreement.

Cryptographic correctness does not repair an incorrect authorization decision. Even assuming that signatures, certificates, token checks, and the authenticated transport work as intended, those mechanisms do not establish that a credential was made available in accordance with the user’s policy. This is a separation of obligations, not a claim that every policy mismatch survives all later checks or that the entire protocol has been exercised.

4.5 Security Consequences

For the rule-selection flaw, source inspection directly establishes that a documented denial can become a positive implementation decision. Because `access()` uses that value for its policy check, the mismatch changes whether execution proceeds towards allocation. The successful allocation branch returns contact material. Conditional on the endpoint’s other checks and ordinary resource requirements succeeding, the mismatch can therefore reach the Provider-grant boundary. This is a conditional control-flow conclusion, not a report of an experimentally observed grant.

In the terminology of Chapter 3, that boundary would constitute a Level 1 violation when

$$\neg \text{Allow}_{\text{spec}}(P_R, I) \wedge \text{ProviderGrant}(I, R).$$

An incorrect positive matcher result alone is not this event: the material

must actually be released. The distinction matters because a policy mismatch can coexist with an unsuccessful endpoint request.

A Level 2 consequence would additionally require

$$\neg \text{Allow}_{\text{spec}}(P_R, I) \wedge \text{TokenIssued}(R, I, T).$$

The shared matcher means that the receiver’s policy precheck need not correct the earlier decision, making propagation relevant to investigate. Nevertheless, token generation occurs after additional checks and depends on valid protocol state. The static mismatch neither satisfies those checks by itself nor records a token being released. It is therefore insufficient as standalone evidence of credential issuance.

Likewise, the Level 3 condition

$$\neg \text{Allow}_{\text{spec}}(P_R, I) \wedge \text{MessageAccepted}(R, I, T, m)$$

requires the receiving application boundary to be reached with an accepted message and valid credential state. Entering a credential-handling branch, generating a token, and accepting a protected message are distinct observations. This chapter establishes no execution exhibiting this final condition.

For the zero-budget discrepancy, the supported conclusion is narrower. The Agent precheck disagrees with the baseline denial at zero, but the Provider’s own decision is unchanged: it rejects that value. The discrepancy alone therefore does not produce a Provider grant on the fresh-contact path. Neither token issuance nor message acceptance follows from the receiver’s failure to reject at this single check. Without additional execution evidence, this finding is a latent authorization inconsistency, not an independently confirmed access violation.

The two findings thus establish concrete policy-evaluation and enforcement disagreements, with different implications for the intermediate boundaries. They show why the authorization-soundness obligations of Chapter 3 cannot be assumed from the implementation’s cryptographic checks. An implementation mismatch is not automatically an end-to-end exploit. Chapter 5 examines concrete execution paths and the evidence needed to determine which, if any, of the Provider-grant, credential-issuance, and message-acceptance boundaries each inconsistency reaches.

5 Attack Construction

5.1 Attack Goal

5.2 Attacker Capabilities

5.3 Rule-Ordering Attack

5.4 Zero-Budget Semantic Attack

5.5 Why Existing Cryptographic Protections Do Not Prevent the Attack

6 Formalization

6.1 Limitations of the Original Model

The registered initiating agent in Chapter 3 should not be silently identified with the attacker in the original symbolic model. That model provides an active network attacker over public communication and uses idealized cryptography, while its fixed peer processes execute their prescribed roles. Its authentication and token-secrecy queries do not include a contact rulebook or link a policy decision to credential use.

The Stage-1 authorization diagnostic preserves those assumptions and adds policy events without making policy evaluation an execution guard. In that diagnostic, a deny event is recorded but does not block the protocol, and the allow event is declared but never executed. The recorded failure of its added correspondence therefore identifies a missing modeled authorization dependence while the baseline authentication and secrecy results are retained. It does not by itself establish a matcher-dependent execution by a malicious registered agent in the implementation.

In particular, the diagnostic event named **Accept** denotes the initiating peer's successful receipt and verification of a token. It does not denote the receiving peer's acceptance of an application message. Its failed correspondence is thus evidence about authorization coverage at token receipt, not a violation of Chapter 3's message-level condition. A message-level claim requires an event at the receiving agent's message-acceptance point and a model connecting policy evaluation, credential issuance, and that event.

6.2 Authorization Events

6.3 Authorization Security Property

6.4 Vulnerable Model

6.5 Repaired Model

7 Evaluation

7.1 Research Questions

7.2 Implementation Validation

7.3 Formal Verification

7.4 Regression of Existing Properties

7.5 Performance and Compatibility

8 Discussion

8.1 Why Authentication Proofs Are Insufficient

8.2 Formal Verification Coverage

8.3 Policy Semantics as Security-Critical State

8.4 Generalization Beyond SAGA

8.5 Limitations

9 Related Work

9.1 Agent Security and Governance

9.2 Capability and Token-Based Authorization

9.3 Formal Verification of Authentication Protocols

9.4 Formal Verification of Authorization Policies

9.5 Authentication–Authorization Gaps

10 Conclusion